HW10-PartB_Adam_Nelson-Archer

April 9, 2026

1 HW10 Part B :: Object Detection with Python

COSC 6373 – Adam Nelson-Archer, 2140122

[1]:
```
!pip install -q transformers torch torchvision Pillow matplotlib requests
```

```
[notice] A new release of pip is available: 24.3.1 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

[2]:
```
import os
import torch
import requests
from PIL import Image, ImageDraw, ImageFont
import matplotlib.pyplot as plt
from transformers import Owlv2Processor, Owlv2ForObjectDetection
```

1.0.1 1. Detect and localize all person-like objects in an image using open-vocabulary prompts.

We'll use the `google/owlv2-base-patch16-ensemble` model from HuggingFace.

[3]:
```
# Load the OWLv2 processor and model
model_name = "google/owlv2-base-patch16-ensemble"
processor = Owlv2Processor.from_pretrained(model_name)
model = Owlv2ForObjectDetection.from_pretrained(model_name)

print("Model and processor loaded successfully.")
```

```
Model and processor loaded successfully.
```

1.0.2 2. Load an image you find online from a URL.

[4]:
```
# Load an image from a URL
image_url = "https://images.unsplash.com/photo-1517649763962-0c623066013b?
↪q=80&w=2070&auto=format&fit=crop"
req = requests.get(image_url, headers={'User-Agent': 'Mozilla/5.0'},␣
↪stream=True)
image = Image.open(req.raw).convert("RGB")
```

```
plt.figure(figsize=(10, 6))
plt.imshow(image)
plt.axis("off")
plt.title("Original Image")
plt.show()
```

Original Image

1.0.3 3. Run OWLv2 with a list of text prompts

Prompts: `["person", "man", "woman", "people", "girl", "boy", "human"]`

1.0.4 4. Apply a confidence threshold (e.g., 0.20) to filter detections.

1.0.5 5. Keep the number of detections to a fixed value (e.g., 15).

[5]:
```
texts = [["person", "man", "woman", "people", "girl", "boy", "human"]]
inputs = processor(text=texts, images=image, return_tensors="pt")

# Run inference
with torch.no_grad():
    outputs = model(**inputs)
```

```
# Target sizes for bounding box un-normalization
target_sizes = torch.tensor([image.size[::-1]])

# Post-process results
confidence_threshold = 0.30
results = processor.post_process_object_detection(outputs=outputs,␣
↪target_sizes=target_sizes, threshold=confidence_threshold)[0]

# Filter and extract detections
boxes = results["boxes"].tolist()
scores = results["scores"].tolist()
labels = results["labels"].tolist()

# Pair them up and sort by confidence score (descending)
detections = list(zip(boxes, scores, labels))
detections.sort(key=lambda x: x[1], reverse=True)

# Keep the number of detections to a fixed value (e.g., 15)
max_detections = 15
detections = detections[:max_detections]

print(f"Found {len(detections)} detections above the threshold.")
```

```
Found 11 detections above the threshold.
```

1.0.6 6. Expand each bounding box slightly using a padding ratio.

1.0.7 7. Save each detected region as a cropped image to a `person_crops/` folder.

a. Draw green bounding boxes and label them on the original image.

[6]:
```
# Configuration
padding_ratio = 0.05
output_folder = "person_crops"
import shutil
shutil.rmtree(output_folder, ignore_errors=True)
os.makedirs(output_folder, exist_ok=True)

# Make a copy of the image to draw on
image_with_boxes = image.copy()
draw = ImageDraw.Draw(image_with_boxes)

# Optional: try to load a default font
try:
    font = ImageFont.truetype("arial.ttf", 24)
except IOError:
    font = ImageFont.load_default()

for i, (box, score, label_idx) in enumerate(detections):
```

```
    xmin, ymin, xmax, ymax = box

    # Calculate padding based on width and height
    width = xmax - xmin
    height = ymax - ymin
    pad_x = width * padding_ratio
    pad_y = height * padding_ratio

    # Apply padding and clamp to image boundaries
    new_xmin = max(0, xmin - pad_x)
    new_ymin = max(0, ymin - pad_y)
    new_xmax = min(image.width, xmax + pad_x)
    new_ymax = min(image.height, ymax + pad_y)

    # Crop and save
    crop_box = (new_xmin, new_ymin, new_xmax, new_ymax)
    cropped_img = image.crop(crop_box)
    crop_filename = os.path.join(output_folder, f"person_crop_{i+1:02d}_{score:.
↪2f}.jpg")
    cropped_img.save(crop_filename)

    # Draw green bounding box and label on the original image
    draw.rectangle(crop_box, outline="green", width=4)

    text_label = f"{texts[0][label_idx]} ({score:.2f})"
    draw.text((new_xmin, new_ymin - 25), text_label, fill="green", font=font)

print(f"Cropped images saved to '{output_folder}/'")

# Show the annotated image
plt.figure(figsize=(12, 8))
plt.imshow(image_with_boxes)
plt.axis("off")
plt.title("Image with Detections")
plt.show()
```

```
Cropped images saved to 'person_crops/'
```

Image with Detections

1.0.8 8. Deliverables

Displaying the saved person crop images.

[7]:
```
# Show up to 15 cropped images
crop_files = sorted([f for f in os.listdir(output_folder) if f.endswith(".
↪jpg")])

if crop_files:
    num_crops = len(crop_files)
    cols = 5
    rows = (num_crops // cols) + (1 if num_crops % cols != 0 else 0)

    plt.figure(figsize=(15, 3 * rows))
    for i, file in enumerate(crop_files):
        img_path = os.path.join(output_folder, file)
        crop_img = Image.open(img_path)

        try:
            score_str = file.split('_')[-1].replace('.jpg', '')
            score = float(score_str)
            title = f"Crop {i+1} ({score:.2f})"
```

```
        except ValueError:
            title = f"Crop {i+1}"

        plt.subplot(rows, cols, i + 1)
        plt.imshow(crop_img)
        plt.axis("off")
        plt.title(title)

    plt.tight_layout()
    plt.show()
else:
    print("No crops found.")
```

1.0.9 11. Discussion: Effects of changing confidence threshold and padding ratio

Based on the detections produced by the OWLv2 open-vocabulary pipeline on the provided crowd image, we can observe the direct impact of the threshold and padding parameters:

Confidence Threshold (`threshold = 0.30`):

By setting a strict confidence threshold of `0.30`, the model effectively filtered out background noise and heavily occluded individuals deep in the crowd, leaving exactly 11 distinct, high-confidence detections.

Originally, I ran this with a threshold of `0.20`, and it produced 15 detections. Two of these

detections were not of people, and were bounding boxes around background noise. I then raised the threshold to `0.40`, and it returned 6 detections. These detections were good, but it ignored a lot of the expected results in the image. Using a threshold of `0.30` produced the best results.

Padding Ratio (`padding_ratio = 0.05`):

Adding a 5% expansion to the bounding boxes proved crucial for capturing full contextual portraits. Object detection models frequently draw bounding boxes that tightly clamp to the visible torso/face.

I also ran this with no padding (`padding_ratio = 0`), and the resulting crops more aggressively sliced off the edges of subjects' hair, hats, or limbs, making them look unnatural.

With excessive padding (e.g., `0.25`): The boxes would expand so far outward that adjacent individuals in this dense crowd would heavily bleed into the crop of the primary subject, ruining the isolation of the target person.

1.1 Acknowledgment

I used a GPT-5.3-Codex to help scaffold and organize this notebook.

Gemini-3.1 was used to check the result and validate conformity with the assignment outline.

All observations and responses were written by me, Adam Nelson-Archer.